

Reviewer Pack — Minimal p-Adic Valuation Degree and SAT Clause Set Entropy C...

Entry 372d6792b89f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 04:46:05 UTC

Minimal p-Adic Valuation Degree and SAT Clause Set Entropy Correlation

Entry ID: 372d6792b89f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 04:46:05 UTC

1. Conjecture

Field A (mathematical branch): p-adic Number Theory **Field B** (complexity object): SAT Clause Sets

Statement:

For every satisfiability problem instance with n variables and m clauses, the minimal p-adic valuation degree of the set of clause indicators is linearly correlated with the entropy of the clause set, such that $\min_{\varphi}(\text{Val}(\varphi)) = \Theta(H(\varphi))$, where $\text{Val}(\varphi)$ denotes the minimum p-adic valuation degree among all clauses in φ , and $H(\varphi)$ represents the Shannon entropy of the clause set.

Rationale (proposer's reasoning):

p-adic valuations have been shown to encode subtle arithmetic properties that could potentially reflect the complexity of logical structures. By linking these to the entropy of clause sets, we may uncover a new perspective on the intrinsic complexity of SAT instances.

Taxonomy category: Arithmetic Complexity Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 49381a52da9cf931

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the generated SAT instances exhibit a correlation coefficient between p-adic valuation degree and clause set entropy greater than or equal to 0.7, with no instance having a p-adic valuation degree exceeding 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_instance(n, m):
    instance = []
    for _ in range(m):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        instance.append(clause)
    return instance

def compute_p_adic_valuation(instance):
    valuation = float('inf')
    for clause in instance:
        product = 1
        for literal in clause:
            if literal == -1:
                product *= -1
            elif literal == 1:
                continue
            else:
                raise ValueError("Invalid literal in clause")
        valuation = min(valuation, abs(product))
    return valuation

def calculate_entropy(clause_set):
    n = len(clause_set)
    counts = [0] * (n + 1)
    for clause in clause_set:
        counts[len(clause)] += 1
    entropy = 0.0
    for count in counts:
        if count > 0:
            p = Fraction(count, n)
            entropy -= p * math.log2(p)
    return entropy
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        m = random.randint(2*n, 3*n)
        instance = generate_random_instance(n, m)
        valuation = compute_p_adic_valuation(instance)
        entropy = calculate_entropy(instance)
        results.append((n, m, valuation, entropy))

    n_max = max(n for n, _, _, _ in results)
    if n_max < 16:
        return {
            "metric_name": "min_φ(Val(φ)) vs H(φ)",
            "metric_value": None,
            "instances_tested": len(results),
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "n_max < 16"
        }

    correlation = 0.0
    for _, _, valuation, entropy in results:
        if valuation > 10:
            continue
        correlation += (valuation - entropy) ** 2

    mean_valuation = sum(valuation for _, _, valuation, _ in results) / len(results)
    mean_entropy = sum(entropy for _, _, _, entropy in results) / len(results)

    if correlation == 0.0:
        return {
            "metric_name": "min_φ(Val(φ)) vs H(φ)",
            "metric_value": None,
            "instances_tested": len(results),
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "No correlation found"
        }

    return {
        "metric_name": "min_φ(Val(φ)) vs H(φ)",
        "metric_value": mean_valuation * mean_entropy / correlation,
        "instances_tested": len(results),
        "n_max": n_max,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []

    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
else:
    first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conje
    print(f"RESULT: FALSIFIED counterexample=\"No correlation found\" first_failing_seed={first

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_850155b3.py", line 106, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_850155b3.py", line 57, in run_trial
    valuation = compute_p_adic_valuation(instance)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_850155b3.py", line 35, in compute_p_adic_valuation
    raise ValueError("Invalid literal in clause")
ValueError: Invalid literal in clause

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its intended task of verifying the conjecture. | next: Re-run the test with proper error handling to ensure that it can complete without crashing and provide the necessary data for verification.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18878
2	preregistration	ollama_remote	glm4:latest	0	0	17643
3	novelty	ollama_remote	glm4:latest	0	0	13007
4	novelty	ollama_remote	glm4:latest	0	0	11621
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14117
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15657
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11712
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17583
9	judge	ollama_remote	glm4:latest	0	0	8632

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 128850 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/372d6792b89f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/372d6792b89f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/372d6792b89f.tar.gz` (if generated)